

# Storing Data in a MongoDB Database

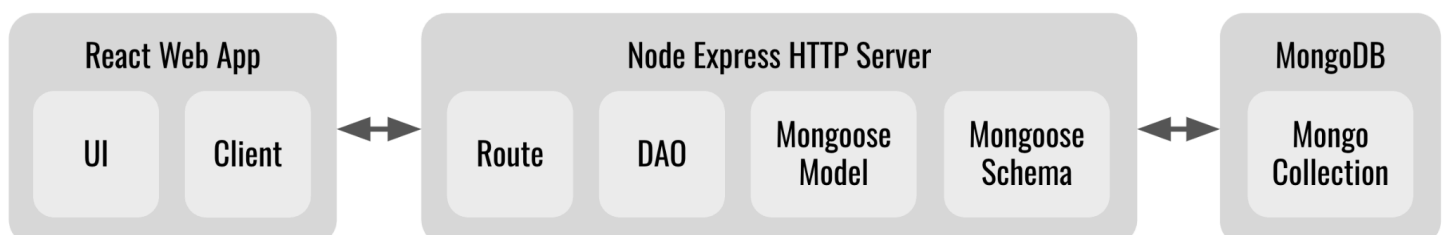
## 1 Introduction

There are two main categories of databases: **relational databases** and **non-relational databases**. Relational databases such as **MySQL**, **SQL Server**, and **Postgre**, store data in **tables** containing **records** of the same **type**, e.g., a table called **courses** would contain records representing all the **courses**, and a **users table** would contain all the **users** of an application. Records are represented as **rows** in the tables where each **column** stores data for attributes specific to the type of the table, e.g., the rows in the **courses** table might have columns such as **name**, **description**, **startDate**, **endDate**, etc. Some of the columns might refer, or **relate** to other records in other tables such as the **instructor** column in the **courses** table might refer to, or relate to a particular row in the **users** table signifying that that particular user is the **instructor** of that particular course. Rows in one table relating to rows in another table is where **relational** databases get their name. The **structured query language** or **SQL**, is a computer language commonly used to interact with relational databases. The **query** in **SQL** generally means to **ask for**, or **retrieve** data that matches some criteria, often written as a **boolean expression** or **predicate**.

More recently there has been a growing interest in representing and storing data using alternative strategies which have collectively come to be referred to as **non relational databases**, or **NoSQL databases**. Non relational databases such as **MongoDB**, **Firebase**, and **Couchbase**, store their data in **collections** containing **documents** which are roughly analogous to **tables** and **records** in their relational counterparts. The biggest difference though is that the columns, or **fields** in the rows in relational databases generally can only contain **primitive data types**, e.g., simple strings, numbers, dates, and booleans, whereas the fields, or **properties** in non relational documents can be arbitrarily **complex data types**, e.g., strings, numbers, booleans, dates as well as combinations of these in complex objects containing arrays of objects of arrays, etc. The other big difference is that relational databases require the structure, or **schema** of the data to be explicitly described before storing any data, whereas non relational databases do not require predefined schemas. Instead, non relational databases delegate this responsibility to the applications using the database. The **structure**, or schema in relational databases is where **structured query language** gets its name.

In the previous chapter we learned how to create an HTTP server with Node.js and integrated it with a React.js Web user interface application to store the application state on the server. In this chapter we expand on this idea to store the data to **MongoDB**, a popular non relational database. The first section demonstrates how to download, install and use a local instance of the MongoDB database. The next section covers how to use the **Mongoose** library to integrate and program a MongoDB database with a Node.js server application. The final section describes how to deploy the database to **Mongo Atlas**, a remote MongoDB database hosted as a cloud service.

The following figure illustrates the overall architecture of what we'll be building in this chapter. From right to left, we'll first create a MongoDB database called **kanbas** where we'll create several **collections** such as **users**, **courses**, **modules**, **assignments**, etc. We'll use the **Mongoose** library to connect to the database programmatically from a Node server. A Mongoose schema will describe the structure of the collections in the MongoDB database and a Mongoose model will implement generic **CRUD** operations. We'll create higher level functions in **Data Access Objects (DAOs)** that operate on the database, and expose those operations through Express routes as RESTful Web APIs. A React Web app will integrate with the RESTful API through a client that will allow the user interface to interact with the database.

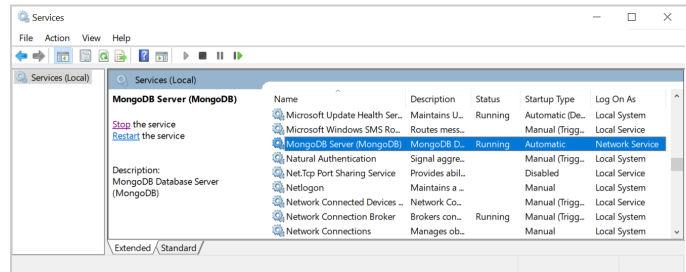


## 2 Working with a Local MongoDB Instance

MongoDB is one of an increasingly popular family of non relational databases. Data is stored in collections of documents usually formatted as JSON objects which makes it very convenient to integrate with JavaScript based frameworks such as Node.js and React.js. This section describes how to install, configure and get started using MongoDB.

### 2.1 Installing and Configuring MongoDB

To get started, [download MongoDB for free](#) selecting the latest version for your operating system, and click **Download**. Run the installer and, if given the choice, choose to **run the database as a service** so that you don't have to bother having to restart the database sever every time you login or restart your computer. The **MongoDB** database will automatically start whenever you start your computer. On Windows, confirm the database is running by searching for **MongoDB** in the **Services** dialog. On macOS, confirm the database is running by clicking the **MongoDB** icon in the **Systems Settings** dialog. The service dialog gives you controls to start and stop the database, but it should already be configured to start automatically when you restart your computer.



#### 2.1.1 Installing MongoDB Manually (optional)

On **macOS** you can install **MongoDB** using **brew** by typing the following at the command line

```
brew install mongodb-atlas
atlas setup
```

Alternatively you can unzip the MongoDB server from the downloaded archive to a local file system and add the right commands to your operating system **PATH** environment variable. On **macOS**, unzip the file into **/usr/local** which creates a directory such as **/usr/local/mongodb-macos-x86\_64-5.0.3** (your version might differ). To be able to execute the database related commands, add the path to the **.bash\_profile** or **.zshrc** file located in your home directory. Add the following line in the configuration file as shown below. Your actual version might differ.

```
~/ .bash_profile or ~/.zshrc

export PATH="$PATH:/usr/local/mongodb-macos-x86_64-5.0.3/bin"
```

If the **.bash\_profile** or **.zshrc** file does not exist in your home directory, create it as a plain text file, but with no extensions and a period in front of it. Configure it as shown above and then **restart your computer**.

On **Windows**, unzip the file into **C:\Program Files**. To configure environment variables on **Windows** press the **Windows + R** key combination to open the **Run** prompt, type **sysdm.cpl** and press **OK**. In the **System Properties** window that appears, press the **Advanced** tab and then the **Environment Variables** button. In the **Environment Variables** configuration window select the **Path** variable and press the **Edit** button. Copy and paste the path of the **bin** directory in the **mongodb** directory you unzipped the MongoDB download, e.g., **"C:\Program Files\mongodb-macos-x86\_64-5.0.3\bin"**. The actual path might differ. Press **OK** and **restart the computer**.

#### 2.1.2 Starting MongoDB from the Command Line

If you installed **MongoDB** as a service, it is already running in the background and can be configured and restarted in **Windows** from the **Services** dialog or from the **System Settings** on **macOS**. Alternatively you can start the MongoDB server from the command line using the **mongod** executable in the **bin** directory where you installed **MongoDB**. First you'll need

to create a **data** folder where the server will store all its data. You can create a data folder in your home directory as shown below.

```
cd ~
mkdir data
```

When you start **MongoDB**, you'll need to tell it where the **data** folder is with the **dbpath** option. If you installed MongoDB on **Windows** in **C:\Program Files\mongodb-macos-x86\_64-5.0.3**, you can start MongoDB from your home directory as shown below.

```
cd ~
C:\Program Files\mongodb-macos-x86_64-5.0.3\bin\mongod --dbpath data
```

Make sure to include the **dbpath data** option to tell **MongoDB** where to find the **data** directory.

If you installed **MongoDB** on **macOS** in **/usr/local/mongodb-macos-x86\_64-6.0.5**, you can start **MongoDB** from your home directory as shown below.

```
cd ~
/usr/local/mongodb-macos-x86_64-6.0.5/bin/mongod --dbpath data
```

## 2.2 Using MongoDB Compass to Interact with MongoDB

Your installation should have installed **MongoDB Compass**, a user interface client to the MongoDB database. If not, [MongoDB Compass can be downloaded from MongoDB's download page](#). You can start **Compass** from your applications folder, or search for it in your operating system's search feature. On **macOS** bring up **Spotlight** by pressing the magnifying glass on the top right menu bar, or press the **Command** (**⌘**) and **Spacebar**. Type MongoDB **Compass** in the search bar and select the application from the result list. On **Windows** press the **Window key** to bring up the search field, type MongoDB **Compass**, and select the application from the result list. When Compass comes up, confirm that the connection string **mongodb://127.0.0.1:27017** appears in the New Connection screen, and press **Connect** to connect to MongoDB.

New Connection

Connect to a MongoDB deployment

URI  Edit Connection String 

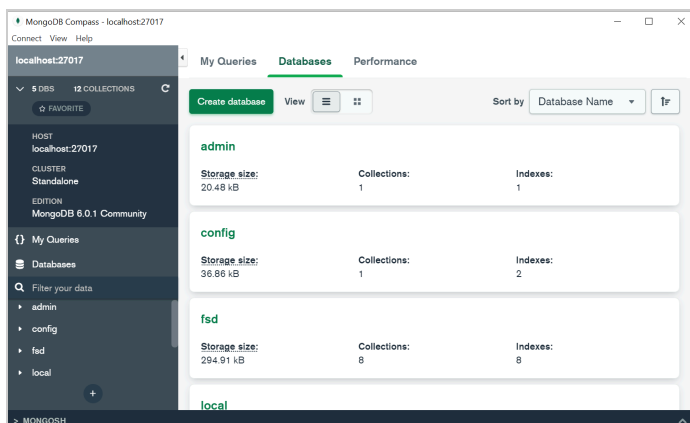
mongodb://localhost:27017

Advanced Connection Options

Save Save & Connect Connect

## 2.3 Creating a MongoDB Database

Once you are connected to a running MongoDB server, click on **Databases** on the left side bar and then click the **Create database** button on the **Databases** tab. In the **Create Database** dialog that appears, name your database **kanbas** and your first collection as **users**. Click **Create Database** to create the **kanbas** database.



Create Database

Database Name

kanbas

Collection Name

courses

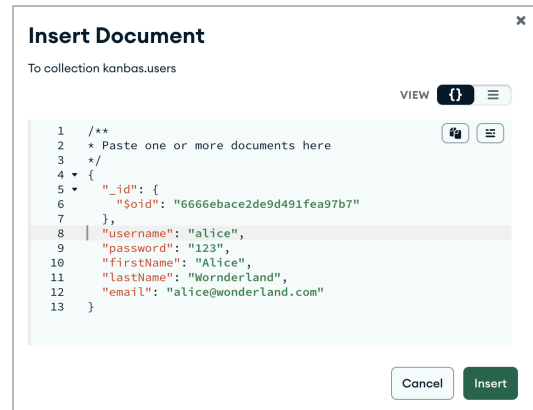
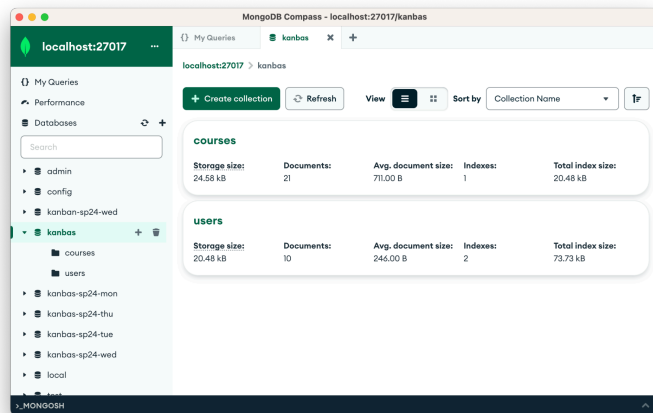
☐ Time-Series  
Time-series collections efficiently store sequences of measurements over a period of time. [Learn More](#)

Additional preferences (e.g. Custom collation, Capped, Clustered collections)

Cancel Create Database

## 2.4 Inserting and Retrieving Data with Compass

In MongoDB, data is organized into **collections**, which are analogous to **tables** in relational databases. Data contained in collections are referred to as **documents**, which are analogous to **records** in relational databases. To create, or **insert** documents into a collection in a MongoDB database using Compass, select the database on the left sidebar and then select the collection you want to insert documents into. For instance, select the **kanbas** database and then the **users** collection as shown below on the right. On the right side, selected **Add Data** and then **Insert Document**. In the **Insert to Collection kanbas.users** dialog that appears, insert the document as shown below. Click **Insert** to insert the document. Confirm the document inserted as expected.



You can also import entire JSON files containing data. Import the [courses.json](#) file we used in earlier assignments under the **Database** directory of your React project. To import click **ADD DATA**, then **Import JSON or CSV file**. Navigate to the location of **courses.json**, select the file and click **Import**. Confirm the courses are imported. Also create the following collections and import the JSON files linked to each of the collection names. Confirm all collections are imported: [modules.json](#), [assignments.json](#), [users.json](#)

Note the objects stored in the database have a primary key **\_id** automatically added by MongoDB when they were inserted. MongoDB primary keys are of type **ObjectId** and are created automatically by the database so your **\_id** values will differ from the ones shown in this document.

## 2.5 Interacting with a MongoDB Database with the Command Line (optional)

Compass is a great graphical user interface to the MongoDB database, but there is also value to knowing how to interact with the database through a command line interface. At the bottom of the **Compass** window there's a **\_MONGOSH** window you can expand to type commands to the database. Let's practice a few commands to retrieve data on the command line. First select the database we want to interact with.

```
> use kanbas
switched to db kanbas
```

All the documents in a collection can be retrieved using the **find()** command on a collection as shown below.

```
> db.courses.find();
```

Documents in a collection can be retrieved by pattern matching their properties. The example below illustrates how to retrieve documents by pattern matching their primary key **\_id**, that is, retrieving the document whose **\_id** field matches **ObjectId('6370104926906053f1597ce6')**. Your ID will likely be different.

```
> db.courses.find({_id: ObjectId("654e8c73ea7ead465908d1cc")})
```

```
{
  _id: ObjectId("654e8c73ea7ead465908d1cc"),
  name: 'Web Development',
  number: 'CS4550',
  startDate: '2023-01-10',
  endDate: '2023-05-15',
  department: 'K123',
  credits: 4
}
```

We can also pattern match any of the other fields individually or combined with other fields. The following example retrieves a document from the **courses** collection whose **number** property is equal to **RS4560**.

```
> db.courses.find({number: 'RS4560'})
{
  _id: 'RS102',
  name: 'Aerodynamics',
  number: 'RS4560',
  startDate: '2023-01-10',
  endDate: '2023-05-15'
}
```

Here's another example retrieving **courses** in the **D134** department.

```
> db.courses.find({department: 'D134'})
{ _id: 'CH101',
  name: 'Organic Chemistry', number: 'CH1230',
  startDate: '2023-01-10', endDate: '2023-05-15',
  department: 'D134', credits: 3
}
{ _id: 'CH102',
  name: 'Inorganic Chemistry', number: 'CH1240',
  startDate: '2023-01-10', endDate: '2023-05-15',
  department: 'D134', credits: 3
}
{ _id: 'CH103',
  name: 'Physical Chemistry', number: 'CH1250',
  startDate: '2023-01-10', endDate: '2023-05-15',
  department: 'D134', credits: 3
}
```

## 3 Programming with a MongoDB database

In the previous section we practiced interacting with the MongoDB database through the Compass graphical interface as well as manually on the command line with **MONGOSH**. This is all and good to make occasional simple queries to confirm the data behaves as expected, but to create applications we're going to need to interact with the database programmatically with libraries such as **Mongoose**. The following sections describe how to install, configure, and connect a Node.js application to a MongoDB database server using the **Mongoose** library. The final section discusses how to configure the application to integrate to a MongoDB database hosted in the Atlas cloud service. Do all your work in a new GitHub branch called **a6** in both your React.js and Node.js projects.

### 3.1 Installing and Connecting to a MongoDB Database

The **Mongoose** library provides a set of operations and abstractions that enhance a MongoDB database and leverages the familiarity of the MONGOSH command line client. To use the Mongoose library, install it from the root of the Node.js project as shown below.

```
$ npm install mongoose
```

To connect to the database server programmatically, import the Mongoose library and then use the **connect** function as shown below. The URL in the **connect** function is called the **connection string** and is currently referring to a MongoDB server instance running in the **localhost** machine (your current laptop or desktop) listening at port **27017** and the **kanbas** database existing in that server. In a later section we'll revisit the connection string and configure it to connect to a database server running in a remote machine hosted by Mongo's Atlas cloud service.

*index.js*

```
import express from "express";
import mongoose from "mongoose";
...
const CONNECTION_STRING = "mongodb://127.0.0.1:27017/kanbas"
mongoose.connect(CONNECTION_STRING);
const app = express();
...
```

*// Load the mongoose Library*

*// connect to the kanbas database*

## 3.2 Configuring Connection Strings as Environment Variables

Instead of hard coding the **connection string** in the source code, it's better to configure it as an environment variable and then reference it from the code. This will come in handy when the server application is deployed to a remote service such as **Render** or **Heroku** and the **connection string** can be configured to reference the online remote database running on Atlas cloud service. In a new **.env** file, declare the following connection string environment variable.

*.env*

MONGO\_CONNECTION\_STRING=mongodb://127.0.0.1:27017/kanbas

Install the **dotenv** library to read configurations in the local environment.

```
$ npm install dotenv
```

Then in **index.js**, import the **dotenv** library to read the **connection string** as shown below.

*index.js*

```
import "dotenv/config";
import express from "express";
import mongoose from "mongoose";
...
const CONNECTION_STRING = process.env.MONGO_CONNECTION_STRING || "mongodb://127.0.0.1:27017/kanbas"
mongoose.connect(CONNECTION_STRING);
const app = express();
app.use(cors());
app.use(express.json());
...
app.listen(process.env.PORT || 4000);
```

## 3.3 Implementing Mongoose Schemas and Models

Now that we have the collections setup in our database, let's now discuss how to connect and interact with the collections in the database using the **Mongoose** library. We'll create Mongoose **Schemas** and **Models** so that we can connect and interact to the database programmatically.

As mentioned earlier, non relational database do not require specifying the structure, or schema of the data stored in collections like relational databases do. That responsibility has been delegated to the applications using non relational databases. Mongoose **schemas** describe the structure of the data being stored in the database and it's used to validate the data being stored or modified through the Mongoose library. The **schema** shown below describes the structure for the **users** collection imported earlier. Create the schema in a **Users** directory in your Node.js projects.



#### Kanbas/Users/schema.js

```
import mongoose from "mongoose";
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  firstName: String,
  email: String,
  lastName: String,
  dob: Date,
  role: {
    type: String,
    enum: ["STUDENT", "FACULTY", "ADMIN", "USER"],
    default: "USER",
  },
  loginId: String,
  section: String,
  lastActivity: Date,
  totalActivity: String,
}, { collection: "users" });
export default userSchema;
```

```
// Load the mongoose library
// create the schema
// String field that is required and unique
// String field that is required but not unique
// String fields
// with no additional
// configurations
// Date field with no configurations

// String field
// allowed string values
// default value if not provided

// store data in "users" collection
```

## 3.4 Implementing Mongoose Models

In earlier sections we demonstrated using the command line client to interact manually with the MongoDB server using the **find** command. Mongoose **models** provide similar functionality to interact with MongoDB programmatically instead of manually. The functions are similar to the ones found in the mongo shell client: **find()**, **create()**, **updateOne()**, **removeOne()**, etc. In **Users/model.js** below, create a Mongoose model from the users schema. The functions provided by Mongoose models are deliberately generic because they can interact with any collection configured in the schema. In the next section we'll create a **data access object** that implements higher level functions specific to the domain of **kanbas**.

#### Kanbas/Users/model.js

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("UserModel", schema);
export default model;
```

```
// Load mongoose library
// Load users schema
// create mongoose model from the schema
// export so it can be used elsewhere
```

## 3.5 Retrieving data from Mongo with Mongoose

The Mongoose model created in the previous section provides low level functions such as **find**, **create**, **updateOne**, and **deleteOne**, that are deliberately vague since they need to be able to operate on any collection. It is good practice to wrap these low level generic functions into higher level functions that are specific to the use cases of the specific projects. For instance instead of just using the generic **find()** function, we'd prefer something such as **findUsers()**, **findUserById()** or **findUserByUsername()**. A previous assignment implemented a **data access object** using arrays declared in files. This assignment refactors the **DAOs** so they use an actual database. The following **Users/dao.js** re-implements the **CRUD** operations for the **users** collection written in terms of the low level Mongoose model operations.

#### Kanbas/Users/dao.js

```
import model from "../model.js";
import db from "../Database/index.js";
export const createUser = (user) => {} // implemented later
export const findAllUsers = () => model.find();
export const findUserById = (userId) => model.findById(userId);
export const findUserByUsername = (username) => model.findOne({ username: username });
export const findUserByCredentials = (username, password) => model.findOne({ username, password });
export const updateUser = (userId, user) => model.updateOne({ _id: userId }, { $set: user });
export const deleteUser = (userId) => model.deleteOne({ _id: userId });
```

## 3.6 Implementing APIs to interact with MongoDB from a React client application

DAOs implement an interface between an application and the low level database access, providing a high level API to the rest of the application hiding the details and idiosyncrasies of using a particular database vendor. Likewise routes implement an interface between the HTTP network world and the JavaScript object and function world by converting a stream of bits from a network connection request into a set of objects, maps, and function event handlers that participate in the client/server architecture of a multi tiered application.

### 3.6.1 Refactoring Account Routes

Previous assignments implemented account routes such as **signin** and **signup** shown below. Since the **DAO** implementations used data structures imported from the local file system, the operations were synchronous. Now that the **DAO** is interacting with a database, the operations are asynchronous and must be tagged with the **async/await** keywords as shown below. Confirm **Signin**, **Signup**, and **Profile** screens work as before. Following the examples below for the **signin** and **signup** functions, add the **async** keyword to all other router functions and add the **await** keyword to all calls to **dao** functions.

Kanbas/Users/routes.js

```
import * as dao from "../dao.js";
export default function UserRoutes(app) {
  const signin = async (req, res) => {
    const { username, password } = req.body;
    const currentUser = await dao.findUserByCredentials(username, password);
    if (currentUser) {
      req.session["currentUser"] = currentUser;
      res.json(currentUser);
    } else {
      res.status(401).json({ message: "Unable to login. Try again later." });
    }
  };
  const signup = async (req, res) => {
    const user = await dao.findUserByUsername(req.body.username);
    if (user) {
      res.status(400).json({ message: "Username already taken" });
      return;
    }
    const currentUser = await dao.createUser(req.body);
    req.session["currentUser"] = currentUser;
    res.json(currentUser);
  };
  ...
  app.post("/api/users/signin", signin);
  app.post("/api/users/signup", signup);
}
```

### 3.6.2 Retrieving All Documents from MongoDB with Mongoose

**DAOs** implement high level data operations based on lower level **Mongoose** models. The **Mongoose** model **find** function retrieves all documents from a collection. The **findAllUsers** function below uses **find** to retrieve all the users from the users collection.

Kanbas/Users/dao.js

```
import model from "../model.js";
export const findAllUsers = () => model.find();
```

**Routes** implement RESTful Web APIs that user interface clients can use to interact with server functionality. The route implemented below uses the **findAll** function implemented by the DAO to retrieve all the users from the database. The route responds with the collection of users retrieved from the database. Confirm the route works by navigating to <http://localhost:4000/api/users> with your browser.



#### Kanbas/Users/routes.js

```
import * as dao from "../dao.js";
let currentUser = null;
export default function UserRoutes(app) {
  const findAllUsers = async (req, res) => {
    const users = await dao.findAllUsers();
    res.json(users);
  };
  app.get("/api/users", findAllUsers);
  ...
}
```

Meanwhile in the React user interface application, in **src/Kanbas/Account/client.ts**, implement the **findAllUsers** function shown below to send a GET request request to the server and **await** for the server's **response** containing an array of users in the **data** property.

#### src/Kanbas/Account/client.ts

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
export const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
export const USERS_API = `${REMOTE_SERVER}/api/users`;
export const findAllUsers = async () => {
  const response = await axiosWithCredentials.get(USERS_API);
  return response.data;
};
...
```

To display the array of users from the database, refactor the **PeopleTable** component to accept an optional **users** parameter, instead of retrieving the users from the local file system. Remove any filters since it's best done at the server.

#### src/Courses/People/Table.tsx

```
import React, { useState, useEffect } from "react";
// import * as db from "../Database";
// import { useParams } from "react-router-dom";
export default function PeopleTable({ users = [] }: { users?: any[] }) {
  // const { cid } = useParams();
  // const { users, enrollments } = db;
  return (
    <div id="wd-people-table">
      <table className="table table-striped">
        ...
        <tbody>
          {users
            .filter((usr) => enrollments.some((enrollment) =>
              enrollment.user === usr.id && enrollment.course === cid))
            .map((user: any) => ( ... ))}
          </tbody>
        </table>
      </div>);}
```

Create a new **Users** screen that fetches the users from the database and displays it with the **PeopleTable** component as shown below.

#### src/Kanbas/Account/Users.tsx

```
import { useState, useEffect } from "react";
import { useParams } from "react-router";
import PeopleTable from "../Courses/People/Table";
import * as client from "../client";
export default function Users() {
  const [users, setUsers] = useState<any[]>([]);
  const { uid } = useParams();
  const fetchUsers = async () => {
    const users = await client.findAllUsers();
```

```

    setUsers(users);
  };
  useEffect(() => {
    fetchUsers();
  }, [uid]);
  return (
    <div>
      <h3>Users</h3>
      <PeopleTable users={users} />
    </div>
  );
};

```

Add a new **Users** route to the **Account** screen as shown below.

*src/Kanbas/Account/index.tsx*

```

...
<Routes>
  <Route path="/" element={ <Navigate to={currentUser ? "/Kanbas/Account/Profile" : "/Kanbas/Account/Signin" } /> } />
  <Route path="/Signin" element={ <Signin /> } />
  <Route path="/Profile" element={ <Profile /> } />
  <Route path="/Signup" element={ <Signup /> } />
  <Route path="/Users" element={ <Users /> } />
</Routes>
...

```

Add a **Users** link to the **Accounts Navigation** sidebar that navigates to the **Users** screen only if the logged in user has an **ADMIN** role. To test, use **Compass** to update the **role** of an existing user or create a new user with **ADMIN** role, signin as the **ADMIN** user, and navigate to the **Users** screen. Confirm that all users are displayed.

*src/Account/Navigation.tsx*

```

import { Link } from "react-router-dom";
import { useSelector } from "react-redux";
export default function AccountNavigation() {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  const links = currentUser ? ["Profile"] : ["Signin", "Signup"];
  const active = (path: string) => (pathname.includes(path) ? "active" : "");
  const { pathname } = useLocation();
  return (
    <div id="wd-account-navigation" className="list-group">
      {links.map((link) => (
        <Link key={link} to={` /Kanbas/Account/${link}`} className={`list-group-item ${active(link)}`}> {link} </Link>
      ))}
      {currentUser && currentUser.role === "ADMIN" && (
        <Link to={` /Kanbas/Account/Users`} className={`list-group-item ${active("Users")}`}> Users </Link> )}
    </div>
  );
};

```

Logged in as an ADMIN, navigate to the new **Users** screen and confirm it displays all the users as shown below.

|                                                                                     |              |            |         |         |               |                |
|-------------------------------------------------------------------------------------|--------------|------------|---------|---------|---------------|----------------|
| Profile                                                                             | Users        |            |         |         |               |                |
| Users                                                                               | Name         | Login ID   | Section | Role    | Last Activity | Total Activity |
|  | Tony Stark   | 001234561S | S101    | ADMIN   | 2020-10-01    | 10:21:32       |
|  | Bruce Wayne  | 001234561S | S101    | STUDENT | 2020-10-01    | 10:21:32       |
|  | Steve Rogers | 001234561S | S101    | STUDENT | 2020-10-01    | 10:21:32       |

### 3.6.3 Retrieving Documents by Predicate from MongoDB with Mongoose

In the **User's DAO**, implement **findUsersByRole** that filters the **users** collection by the **role** property as shown below. Mongoose model's **find** function takes as argument a JSON object used to pattern match documents in the collection. The **{role: role}** object means that documents will be filtered by their **role** property that matches the value **role**.

Kanbas/Users/dao.js

```
export const findUsersByRole = (role) => model.find({ role: role }); // or just model.find({ role })
```

In the **User routes**, refactor the **findAllUsers** function so that it parses the **role** from the query string, and then uses the **DAO** to retrieve users with that particular role.

Kanbas/Users/routes.js

```
const findAllUsers = async (req, res) => {
  const { role } = req.query;
  if (role) {
    const users = await dao.findUsersByRole(role);
    res.json(users);
    return;
  }
  const users = await dao.findAllUsers();
  res.json(users);
};
```

In the React user interface application, add **findUserByRole** in the client so that it encodes the **role** in the query string of the URL as shown below.

src/Kanbas/Account/client.ts

```
export const findUsersByRole = async (role: string) => {
  const response = await
    axios.get(`${USERS_API}?role=${role}`);
  return response.data;
};
```

In the **Users** screen, add a dropdown that invokes a **filterUsers** event handler function with the selected **role**. The function updates a **role** state variable and requests from the server the list of users filtered by their role. Confirm that selecting various roles actually filters the users by their role.

src/Kanbas/Account/Users.tsx

```
export default function Users() {
  const [users, setUsers] = useState<any[]>([]);
  const [role, setRole] = useState("");
  const filterUsersByRole = async (role: string) => {
    setRole(role);
    if (role) {
      const users = await client.findUsersByRole(role);
      setUsers(users);
    } else {
      fetchUsers();
    }
  };
};
```

```
const fetchUsers = async () => { ... };
useEffect(() => { ... }, []);
return (
```

```
  <div>
    <h3>Users</h3>
```

```
    <select value={role} onChange={(e) => filterUsersByRole(e.target.value)}
      className="form-select float-start w-25 wd-select-role" >
      <option value="">All Roles</option> <option value="STUDENT">Students</option>
      <option value="TA">Assistants</option> <option value="FACULTY">Faculty</option>
      <option value="ADMIN">Administrators</option>
```

Assistants

Name

Login ID

Section

Role



Natasha Romanoff

001234564S

S101

TA



Aragorn Elessar

001234568S

S101

TA

```

    </select>
    ...
  </div>
);}

```

Now practice filtering users by their **first** or **lastName** by creating a **regular expression** used to pattern match the **firstName** or **lastName** fields of the documents in the **users** collection.

*Kanbas/Users/dao.js*

```

export const findUsersByPartialName = (partialName) => {
  const regex = new RegExp(partialName, "i"); // 'i' makes it case-insensitive
  return model.find({
    $or: [{ firstName: { $regex: regex } }, { lastName: { $regex: regex } }],
  });
};

```

In the the **User routes**, parse a **name** parameter from the **query string** and use it to find users whose first or last names partially match the **name** parameter.

*Kanbas/Users/routes.js*

```

const findAllUsers = async (req, res) => {
  const { role, name } = req.query;
  if (role) { ... }
  if (name) {
    const users = await dao.findUsersByPartialName(name);
    res.json(users);
    return;
  }
  const users = await dao.findAllUsers();
  res.json(users);
};

```

In the React client application, implement the **findUserByPartialName** client function as shown below which encodes a **name** in the **query string** the server can use to filter users by their **first** and **lastName**.

*src/Kanbas/Account/client.ts*

```

export const findUsersByPartialName = async (name: string) => {
  const response = await axios.get(`${USERS_API}?name=${name}`);
  return response.data;
};

```

In the **Users** screen, create a new **name** state variable and corresponding **input** field used to invoke the **findUsersByPartialName** client function and update the **users** state variable with a subset of users that match the **name**. Confirm that typing a name in the input field actually filters the users by their first or last name. Note that the current implementation does not consider a combination of filtering by **role** and by **name**. Feel free to explore how you would go about filtering by both.

*src/Kanbas/Account/Users.tsx*

```

export default function Users() {
  const [users, setUsers] = useState<any[]>([]);
  const [role, setRole] = useState("");
  const [name, setName] = useState("");
  const filterUsersByName = async (name: string) => {
    setName(name);
    if (name) {
      const users = await client.findUsersByPartialName(name);
      setUsers(users);
    } else {
      fetchUsers();
    }
  };
}

```

| rd                                                                                                     | All Roles  |         |         |  |
|--------------------------------------------------------------------------------------------------------|------------|---------|---------|--|
| Name                                                                                                   | Login ID   | Section | Role    |  |
|  Steve Rogers22   | 001234563S | S101    | STUDENT |  |
|  Natasha Romanoff | 001234564S | S101    | TA      |  |
|  Frodo Baggins    | 001234567S | S101    | FACULTY |  |

```

};
...
return (
  <div>
    <h3>Users</h3>
    <input onChange={e => filterUsersByName(e.target.value)} placeholder="Search people"
      className="form-control float-start w-25 me-2 wd-filter-by-name" />
    ...
  </div>
);
}

```

### 3.6.4 Retrieving Documents by Primary Key from MongoDB with Mongoose

A common database operation is to retrieve documents by their primary key. The **DAO** function below retrieves a **user** document by its primary key.

*Kanbas/Users/dao.js*

```
export const findUserId = (userId) => model.findById(userId);
```

Make the **findUserId** DAO function available as a RESTful Web API as shown below.

*Kanbas/Users/routes.js*

```

const findUserId = async (req, res) => {
  const user = await dao.findById(req.params.userId);
  res.json(user);
};
app.get("/api/users/:userId", findUserId);

```

The user interface can then interact with the server using the **findUserId** client function shown below.

*src/Kanbas/Account/client.ts*

```

export const findUserId = async (id: string) => {
  const response = await axios.get(`${USERS_API}/${id}`);
  return response.data;
};

```

In a new **PeopleDetails** component, use the client's **findUserId** function to retrieve the user when a faculty clicks on the user's name. Parse a **uid** path parameter and use it to retrieve the user by their ID when the component loads. If the **uid** does not exist, return **null** so that the component does not render on the screen. In **useEffect**, add **uid** as a dependency so that if the component re-renders if you click on another user while the component is still displaying.

*src/Kanbas/Courses/People/Details.tsx*

```

import { useEffect, useState } from "react";
import { FaUserCircle } from "react-icons/fa";
import { IoCloseSharp } from "react-icons/io5";
import { useParams, useNavigate } from "react-router";
import { Link } from "react-router-dom";
import * as client from "../../Account/client";
export default function PeopleDetails() {
  const { uid } = useParams();
  const [user, setUser] = useState<any>({});
  const navigate = useNavigate();
  const fetchUser = async () => {
    if (!uid) return;
    const user = await client.findUserId(uid);
    setUser(user);
  };
  useEffect(() => {
    if (uid) fetchUser();
  }, [uid]);
}

```

| Name                                                                                             | Login ID   | Section |
|--------------------------------------------------------------------------------------------------|------------|---------|
|  Tony Stark   | 001234561S | S101    |
|  Bruce Wayne  | 001234562S | S101    |
|  Steve Rogers | 001234563S | S101    |




**Tony Stark**

**Roles:** STUDENT

**Login ID:** 001234561S

**Section:** S101

**Total Activity:** 10:21:32

```

}, [uid]));
if (!uid) return null;
return (
  <div className="wd-people-details position-fixed top-0 end-0 bottom-0 bg-white p-4 shadow w-25">
    <button onClick={() => navigate(-1)} className="btn position-fixed end-0 top-0 wd-close-details">
      <IoCloseSharp className="fs-1" /> </button>
    <div className="text-center mt-2"> <FaUserCircle className="text-secondary me-2 fs-1" /> </div><hr />
    <div className="text-danger fs-4 wd-name"> {user.firstName} {user.lastName} </div>
    <b>Roles:</b> <span className="wd-roles"> {user.role} </span> <br />
    <b>Login ID:</b> <span className="wd-login-id"> {user.loginId} </span> <br />
    <b>Section:</b> <span className="wd-section"> {user.section} </span> <br />
    <b>Total Activity:</b> <span className="wd-total-activity">{user.totalActivity}</span> </div> ); }

```

Add a **close button** rendered as an **X** at the top left that hides the component by navigating back to the **Users** screen as shown above. Confirm that clicking on the the name of a user displays the user's details. Also confirm that closing the **PeopleDetails** hides the component.

src/Kanbas/Courses/People/Table.tsx

```

...
import * as client from "../../Account/client";
import PeopleDetails from "../Details";
export default function PeopleTable() {
  ...
  return (
    <div id="wd-people-table">
      <PeopleDetails />
      ...
      {users
        .map((user) => (
          <tr key={user._id}>
            <td className="wd-full-name text-nowrap">
              <Link to={`../Kanbas/Account/Users/${user._id}`} className="text-decoration-none">
                <FaUserCircle className="me-2 fs-1 text-secondary" />
                <span className="wd-first-name">{user.firstName}</span>{" "}
                <span className="wd-last-name">{user.lastName}</span>
              </Link>
            </td>
            ...
          </tr>
        ))}
      ...
    </div>
  );
}

```

In the **Account** screen, add a **Route** that encodes the **uid** path parameter in the **URL** that renders the same **Users** screen.

src/Kanbas/Account/index.tsx

```

...
<Routes>
  <Route path="/" element={ <Navigate to={currentUser ? "/Kanbas/Account/Profile" : "/Kanbas/Account/Signin" } /> } />
  <Route path="/Signin" element={ <Signin /> } />
  <Route path="/Profile" element={ <Profile /> } />
  <Route path="/Signup" element={ <Signup /> } />
  <Route path="/Users" element={ <Users /> } />
  <Route path="/Users/:uid" element={ <Users /> } />
</Routes>
...

```

## 3.6.5 Deleting a Document in MongoDB with Mongoose

To delete user documents from the **users** MongoDB collection, implement the **deleteUser** operation as shown below. The **DAO** function below removes a single **user** document from the database based on its primary key.

Kanbas/Users/dao.js

```
export const deleteUser = (userId) => model.deleteOne({ _id: userId });
```

The route below makes the **deleteUser** operation available as a RESTful Web API for integration with the user interface which encodes the id of the user to remove as a path parameter.

*Kanbas/Users/routes.js*

```
const deleteUser = async (req, res) => {
  const status = await dao.deleteUser(req.params.userId);
  res.json(status);
};
app.delete("/api/users/:userId", deleteUser);
```

In the React Web App, implement client function that integrates with the **deleteUser** route in the server.

*src/Kanbas/Account/client.ts*

```
export const deleteUser = async (userId: string) => {
  const response = await axios.delete(`${USERS_API}/${userId}`);
  return response.data;
};
...
```

In the **PeopleDetails** component add buttons **Cancel** and **Delete** as shown below. The **Delete** button invokes a new **deleteUser** event handler function with **uid**, the ID of the user to delete. Pass a reference to **fetchUsers** as a parameter so that **PeopleDetails** can notify **PeopleTable** that a user has been remove and that the list of users must be updated.

*src/Kanbas/Courses/People/Details.tsx*

```
...
import { useNavigate, useParams } from "react-router";
export default function PeopleDetails() {
  const navigate = useNavigate();
  const deleteUser = async (uid: string) => {
    await client.deleteUser(uid);
    navigate(-1);
  };
  ...
  return (
    <div className="...">
      ...
      <hr />
      <button onClick={() => deleteUser(uid)} className="btn btn-danger float-end wd-delete" > Delete </button>
      <button onClick={() => navigate(-1)}
        className="btn btn-secondary float-start float-end me-2 wd-cancel" > Cancel </button>
    </div>
  );
};
```

Name

-  Tony Stark
-  Bruce Wayne
-  Steve Rogers
-  Natasha Romanoff

Bruce Wayne 

Roles: STUDENT

Login ID: 001234562S

Section: S101

Total Activity: 15:32:43

Cancel

Delete

Use the **client's deleteUser** to remove the user, and navigate back to hide the details component. The **Cancel** button just hides the details without removing any documents. Confirm that clicking the **Cancel** and **Delete** buttons actually work.

### 3.6.6 Updating a Document in MongoDB with Mongoose

The **Mongoose update** function updates documents in **MongoDB** databases. In the **User's DAO**, implement **updateUser** as shown below to update a single document by first identifying it by its primary key, and then updating the matching fields in the **user** parameter.

*Kanbas/Users/dao.js*

```
import model from "../model.js";
export const updateUser = (userId, user) => model.updateOne({ _id: userId }, { $set: user });
```



```
...
```

In the **User's routes**, make the **DAO** function available as a RESTful Web API as shown below. Map a route that accepts a user's primary key as a path parameter, passes the ID and request body to the DAO function and responds with the status.

*Kanbas/Users/routes.js*

```
export default function UserRoutes(app) {
  ...
  const updateUser = async (req, res) => {
    const { userId } = req.params;
    const userUpdates = req.body;
    await dao.updateUser(userId, userUpdates);
    const currentUser = req.session["currentUser"];
    if (currentUser && currentUser._id === userId) {
      req.session["currentUser"] = { ...currentUser, ...userUpdates };
    }
    res.json(currentUser);
  };
  app.put("/api/users/:userId", updateUser);
  ...
}
```

In the React client application, the client function **updateUser** sends user updates to the server to be saved to the database.

*src/Kanbas/Account/client.ts*

```
...
export const updateUser = async (user: any) => {
  const response = await axiosWithCredentials.put(`${USERS_API}/${user._id}`, user);
  return response.data;
};
...
```

In the **PeopleDetails** component, add a **name** state variable to edit the **first** and **last** name of the user. Also add a **editing** state variable to toggle the input field that edits the **name**. With the **navigate** routing function, navigate back to **PeopleTable** once the update is done. Create a new **saveUser** function that splits the **name** state variable into the **firstName** and **lastName** and sends an updated version of the user to the server. Also update the local **user** state variable, turn off editing, and **navigate** back to the **PeopleTable**.

*src/Kanbas/Courses/People/Details.tsx*

```
...
import { FaPencil } from "react-icons/fa6";
import { FaCheck, FaUserCircle } from "react-icons/fa";
export default function PeopleDetails() {
  ...
  const [name, setName] = useState("");
  const [editing, setEditing] = useState(false);
  const saveUser = async () => {
    const [firstName, lastName] = name.split(" ");
    const updatedUser = { ...user, firstName, lastName };
    await client.updateUser(updatedUser);
    setUser(updatedUser);
    setEditing(false);
    navigate(-1);
  };
  ...
  return (
    ...
    <div className="text-danger fs-4">
      {!editing && (
        <FaPencil onClick={() => setEditing(true)}
          className="float-end fs-5 mt-2 wd-edit" /> )}
      {editing && (
        // to edit the user's first and last name
        // whether we are editing or not
        // to save updates to user's name
        // split the name into an array and get first
        // and last and create new version of user overriding
        // first and last. Send update to server
        // update local copy of the user
        // we're done editing
        // go back to PeopleTable

        // if not editing show pencil icon
        // clicking pencil turns on editing and hides pencil
      )}
    </div>
  );
}
```

```

    <FaCheck onClick={() => saveUser()}
      className="float-end fs-5 mt-2 me-2 wd-save" /> )}
    {!editing && (
      <div className="wd-name"
        onClick={() => setEditing(true)}>
        {user.firstName} {user.lastName}</div>)}
    {user && editing && (
      <input className="form-control w-50 wd-edit-name"
        defaultValue={`${user.firstName} ${user.lastName}`}
        onChange={(e) => setName(e.target.value)}
        onKeyDown={(e) => {
          if (e.key === "Enter") { saveUser(); }}} />)}
    </div>
    ...
  );}

```

*// if editing show check mark. Clicking check turns*  
*// off editing, saves and hides check*  
*// if not editing show first and last name*  
*// clicking on name turns on editing*  
  
*// if editing show input field to edit name*  
  
*// name is initially concatenation of first and last*  
*// update name as we type*  
  
*// save the user if Enter key is pressed*

Add **pencil** and **check** icons to turn **editing** on and off. Hide each icon based on the **editing** boolean state variable. Clicking the name of the user also turns **editing** on. If **editing** is on, hide the user's name and instead display an input field that shows the current user's **firstName** and **lastName** and edits the **name** state variable. Pressing the **Enter** key saves the updated user's details. Confirm users can be edited. On your own, add the ability to edit a user's email and role. Use an input field of type **email** to edit the email. To edit the user's role, use a dropdown similar to the one used to filter users by their role.

### 3.6.7 Creating New Documents in MongoDB with Mongoose

In the **User's DAO**, implement the **createUser** function as shown below to insert a new **user** object into the **users** collection.

*Kanbas/Users/dao.js*

```

export const createUser = (user) => {
  delete user._id
  return model.create(user);
}

```

*// remove \_id field just in case client sends it*  
*// database will create \_id for us instead*

Make sure the incoming **user** object does not have an **\_id** property since it can interfere with the database insert operation. In the **User's routes**, make the DAO operation available as a RESTful Web API for the user interface to interact with. The new user is posted to the route in the request's body. The DAO **createUser** function inserts the new user into the database and returns the newly inserted **user** which is sent back to the user interface in the response.

*Kanbas/Users/routes.js*

```

import * as dao from "../dao.js";
let currentUser = null;
export default function UserRoutes(app) {
  ...
  const createUser = async (req, res) => {
    const user = await dao.createUser(req.body);
    res.json(user);
  };
  app.post("/api/users", createUser);
  ...
}

```

In the React client application, implement a **createUser** client function to interact with the route created above. Post the new user object to the server as shown below.

*src/Kanbas/Account/client.ts*

```

export const createUser = async (user: any) => {
  const response = await axios.post(`${USERS_API}`, user);
  return response.data;
};

```

In the **Users** screen, implement a new **createUser** event handler that sends a new user object to be inserted in the database. Use default values for the fields as shown and confirm that clicking the new **+ People** button actually creates the new user. Optionally implement editing those fields in the **PeopleDetails** component.

src/Kanbas/Account/Users.tsx

```
export default function Users() {
  const { uid } = useParams();
  const [users, setUsers] = useState<any[]>([]);
  const [role, setRole] = useState("");
  const [name, setName] = useState("");
  const createUser = async () => {
    const user = await client.createUser({
      firstName: "New",
      lastName: `User${users.length + 1}`,
      username: `newuser${Date.now()}`,
      password: "password123",
      email: `email${users.length + 1}@neu.edu`,
      section: "S101",
      role: "STUDENT",
    });
    setUsers([...users, user]);
  };
  return (
    <div>
      <button onClick={createUser} className="float-end btn btn-danger wd-add-people">
        <FaPlus className="me-2" />
        Users
      </button>
      ...
    </div>
  );
};
```

## 4 Integrating with MongoDB Hosted in Atlas Cloud Service

When you run your server on your development environment, it should be connecting to a MongoDB instance running on the same local development computer. When you deploy the server on a remote server such as **Render**, **Heroku** or **AWS**, the server needs to connect to a database that is also hosted on a public site. **MongoDB Atlas Cloud Service** provides a hosted database service where a MongoDB instances run on public servers, and they provide a connection string to integrate our Node.js application. This section describes setting up and deploying the database online and then integrating with it from our Node.js server running on Heroku.

Create a database user using a username and password. Users will be given the read and write to any database **privilege** by default. You can update these permissions and/or create additional users later. Ensure these credentials are different to your MongoDB Cloud username and password. You can manage existing users via the [Database Access Page](#).

Username  
giuseppi

Password  
\*\*\*\*\*

Autogenerate Secure Password Copy

Create User

| Username | Authentication Type |
|----------|---------------------|
| giuseppi | Password            |

EDIT REMOVE

### 4.1 Setting up MongoDB Atlas

To get started, head over to <https://www.mongodb.com/> and click on **Sign in** at the top right corner. Login with your **Google** account or click on **Sign Up** to create an account with an email and password. If you get a validation email, confirm it and login. Answer any general questions if asked during the sign up process. In the **Deploy your cluster** screen choose a **Free** plan for now which should be enough for this course. Name your cluster **Kanbas**. In the **Provider** section, choose any of the cloud providers and in the **Region** section choose a region close to your geographic area, for instance **AWS** and **North Virginia**, and then click **Create Deployment**. In the **Connect to Kabas** screen, in the **Create a database user** section create credentials to login to your database. In the **Username** and **Password** fields, type credentials you'll remember later since these are the credentials **mongoose** will use to login to the database from your Node.js server application when running on Render or Heroku. If you forget these credentials you'll

Where would you like to connect from?

Enable access for any network(s) that need to read and write data to your cluster.

My Local Environment Use this to add network IP addresses to the IP Access List. This can be modified at any time.

Cloud Environment Use this to configure network access between Atlas and your cloud or on-premise environment. Specifically, set up IP Access Lists, Network Peering, and Private Endpoints.

Advanced

Add entries to your IP Access List

Only on IP address you add to your Access List will be able to connect to your project's clusters. You can manage existing IP entries via the [Network Access Page](#).

| IP Address       | Description       |                           |
|------------------|-------------------|---------------------------|
| Enter IP Address | Enter description | Add My Current IP Address |
| Add Entry        |                   |                           |

| IP Access List   | Description   |             |
|------------------|---------------|-------------|
| 68.160.144.70/32 | My IP Address | EDIT REMOVE |

Finish and Close

need to create new ones later. I went with **giuseppi** and **supersecretpassword** :) and clicked **Create Database User**.

## 4.1.1 Connecting to a Remote Database from Compass

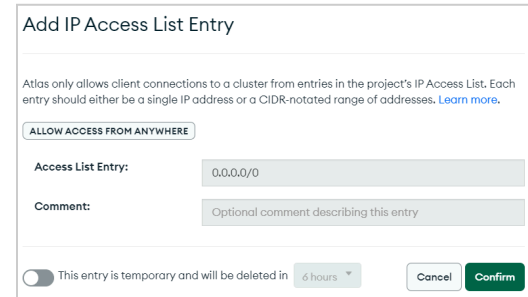
Then click the **Choose a connection method** button, and in the **Access your data through tools**, select **Compass**. In the **Connecting with MongoDB Compass** screen, in the **Copy the connection string, then open MongoDB Compass** section, copy the connection string which should look something like so

```
mongodb+srv://giuseppi:supersecretpassword@kanbas.jxui0bc.mongodb.net/
```

Click **Done**. From **Compass** select **Connect, New Window**, paste the connection string in the **URI** field, and then click connect. Following the same steps used earlier, import the **courses**, **modules**, and **users** into the remote database.

## 4.1.2 Connecting from Node.js

In the **Atlas** main window click on **Network Access** and in the **Network Access** screen **IP Access List** tab, select **+ ADD IP ADDRESS**. In the **Add IP Access List Entry** dialog click on **ALLOW ACCESS FROM ANYWHERE**. This will add **0.0.0.0/0** to the **Access List Entry**, allowing any computer to connect. Click **Confirm** and verify the new entry appears in the **Network Access** screen. Click **Database** on the left and in the **Clusters** screen, click **Connect**. In the **Connect to Kanbas** dialog, in the **Connect to your application**, select **Drivers and version** section, confirm the **Driver** is set to **Node.js** and **Version** is set to **5.5 or later**. In the **Add your connection string into your application code** section, copy the the URL. It should look similar to the following.



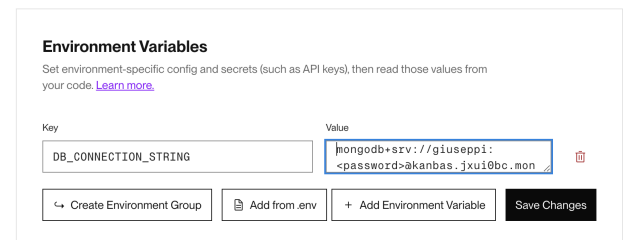
### Node MongoDB Database Connection String

```
mongodb+srv://giuseppi:<password>@kanbas.jxui0bc.mongodb.net/kanbas?retryWrites=true&w=majority&appName=Kanbas
```

Commit and push your code to a branch called **a6** and deploy the server application to a new remote service running on **Render**, **Heroku**, or **AWS**. Make sure not to deploy to the server for prior assignments since TAs might still be grading it. In the new remote server, configure an environment variable called **MONGO\_CONNECTION\_STRING** with the URL value above. For instance, in the **Render** dashboard click on **Environment**, type **MONGO\_CONNECTION\_STRING** in the **Key** field and the URL in the **Value** field. Replace the **<password>** with the actual password created in an earlier step. Commit and deploy the React application to a new **a6** branch. Configure the **REACT\_APP\_REMOTE\_SERVER** to point to the new remote server. For the environment variables to take effect, you might need to redeploy and/or restart the remote Node server on Render as well as the remote React application server on Netlify.

Events  
Logs  
Disks  
Environment  
Shell  
Previews  
Jobs  
Metrics  
Scaling  
Settings

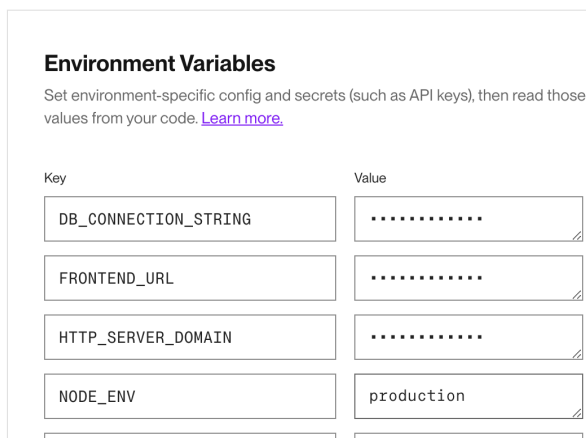
### Environment



## 4.2 Configuring Session in Remote Servers

The local Node server was configured to support multiple sessions. Similarly, the remote server also needs to be configured to support sessions. In **Render.com**, navigate to the **Environment** section of the application dashboard. Click **Add Environment Variable**, type the name of the variables in the **Key** column and the variable's value in the **Value** column. Repeat for each of the environment variables as shown here

Events  
Logs  
Disks  
Environment  
Shell  
Previews  
Jobs  
Metrics  
Scaling  
Settings



on the right. You will need to restart the remote server by clicking **Manual Deploy** and then **Deploy latest commit**. Below is an example of the environment variables used to configure a remote server running on **Render.com**. Use the same **Environment Variable** keys shown on the left column below. Don't use the values shown on the right column below. Instead use the values for your Mongo Database, your Netlify remote server, and your remote Node server.

| Environment Variable    | Value                                                                                                                   |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------|
| MONGO_CONNECTION_STRING | mongodb+srv://giuseppi:supersecretpassword@kanbas.jxui0bc.mongodb.net/kanbas?retryWrites=true&w=majority&appName=Kanbas |
| NETLIFY_URL             | https://a6--kanbas-react-web-app-su24.netlify.app                                                                       |
| NODE_SERVER_DOMAIN      | kanbas-node-server-app-su24.onrender.com                                                                                |
| NODE_ENV                | production                                                                                                              |
| SESSION_SECRET          | what ever                                                                                                               |

## 5 Implementing the Kanbas MongoDB

For the last several assignments we have been implementing the Kanbas application rendering various courses, modules and assignments. We used **JSON** files to represent the data for the application, and wrapped the data structures into a "Database" that first lived in the React application and then in the Node server. It is time for the data to live where it belongs. In this section, on your own, migrate the **courses** and **modules** into corresponding collections in the **kanbas** MongoDB database. Create the **Mongoose** schemas, models and DAOs, and refactor the RESTful Web APIs to **CRUD** courses and modules in the database. Confirm that all **courses** and **modules** **CRUD** functionality works as expected. Optionally also migrate the assignments into a collection and confirm that all assignment **CRUD** operations work. Note that the **modules course** field might be referencing an **ID** of a course that does not exist. Update the **module course** fields so that they refer to the correct **course** document.

### 5.1 Persisting Courses in the Database

The current server application implements RESTful Web APIs to access courses stored in a file **courses.js**. This section replaces the source of the courses in a **MongoDB** **courses** collection. The basic operations on any data source are create, read, update and delete, colloquially referred to as **CRUD** operations. The following sections demonstrate how to implement several **CRUD** operations.

#### 5.1.1 Retrieving Courses from a Database

To access the **courses** collection created earlier, create a **mongoose schema** file as shown below. The schema file describes the **constraints** of the **documents** stored in a **collection**, such as the names of the **properties**, their data types, and the name of the collection where the documents will be stored.

*Kanbas/Courses/schema.js*

```
import mongoose from "mongoose";
const courseSchema = new mongoose.Schema(
  {
    name: String,
    number: String,
    credits: Number,
    description: String,
  },
  { collection: "courses" }
);
export default courseSchema;
```

Using the **mongoose schema** file, create a **mongoose model** file as shown below. Mongoose models provide functions to interact with the collection such as **find()**, **create()**, **updateOne()**, and **deleteOne()**. The **CourseModel** in the mongoose model declares a unique name that can be used as a reference from other mongoose schemas.

Kanbas/Courses/model.js

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("CourseModel", schema);
export default model;
```

Using the **mongoose model**, create a **DAO** file that implements various **CRUD** operations to interact with the **courses** collection. Start by refactoring the **findAllCourses()** function to retrieve all the courses from the database instead of the **Database** file as shown below. The **model.find()** function returns an array containing all the courses documents in the **courses** collection.

Kanbas/Courses/dao.js

```
// import Database from "../Database/index.js";
import model from "../model.js";

export function findAllCourses() {

// return Database.courses;
  return model.find();
}
...
```

The functions in **mongoose models** all return **promises** allowing **asynchronous** communication with the MongoDB server. In the Courses **routes** file, declare all router functions as asynchronous by adding the **async** keyword in front of the router functions as shown below. Also add the **await** keyword in front of all asynchronous calls of the **DAO** functions such as **dao.findAllCourses()** as shown below.

Kanbas/Courses/routes.js

```
import * as dao from "../dao.js";
export default function CourseRoutes(app) {
  app.get("/api/courses", async (req, res) => {
    const courses = await dao.findAllCourses();
    res.send(courses);
  });
  ...
}
```

In the React.js project, refactor the **fetchAllCourses** client function so that it uses a version of **axios** that includes cookies in the request, as shown below. Confirm you can access all courses from the browser: <http://localhost:4000/api/courses>.

src/Kanbas/Courses/client.ts

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;
export const fetchAllCourses = async () => {
  const { data } = await axiosWithCredentials.get(COURSES_API);
  return data;
};
```

In the **Kanbas** root component, import **course client** and refactor the **fetchCourse** function to fetch all the courses, as shown below.

src/Kanbas/index.tsx

```
import { useEffect, useState } from "react";
import * as courseClient from "../Courses/client";
export default function Kanbas() {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  const [courses, setCourses] = useState<any[]>([]);
  const fetchCourses = async () => {
    try {
      const courses = await courseClient.fetchAllCourses();
      setCourses(courses);
    } catch (error) {
      console.error(error);
    }
  };
  useEffect(() => {
    fetchCourses();
  }, [currentUser]);
  return ( ... );
}
```

In the **Dashboard** screen, remove the **filter** since filtering is best implemented on the server. Start the database, server, and client application, sign in on the React application, and navigate to the Dashboard screen. Confirm that all courses are rendered.

src/Kanbas/Dashboard.tsx

```
export default function Dashboard({ ... }) {
  return (
    <div id="wd-dashboard">
      <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
      <div id="wd-dashboard-courses" className="row">
        <div className="row row-cols-1 row-cols-md-5 g-4">
          {courses
            .filter((course) => ( ... ))
            .map((course) => ( ... ))}
        </div>
      </div>
    </div>
  );
}
```

## 5.1.2 Inserting Courses into a Database

Refactor the **DAO's** **createCourse()** function to insert new courses into the database with the **mongoose model** as shown below.

Kanbas/Courses/dao.js

```
import model from "../model.js";
export function createCourse(course) {
  delete course._id;
  return model.create(course);
  // const newCourse = { ...course, _id: Date.now().toString() };
  // Database.courses = [...Database.courses, newCourse];
  // return newCourse;
}
...
```

Refactor the Courses routes file by adding keywords **async** and **await** before the route and DAO functions as shown below.

Kanbas/Courses/routes.js

```
import * as dao from "../dao.js";
export default function CourseRoutes(app) {
  app.post("/api/courses", async (req, res) => {
    const course = await dao.createCourse(req.body);
    res.json(course);
  });
}
```



```
});
...
}
```

In the React.js Courses client file, refactor the **createCourse** client function so that it uses the axios instance with credentials as shown below.

*Kanbas/Courses/client.ts*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;
export const createCourse = async (course: any) => {
  const { data } = await axiosWithCredentials.post(COURSES_API, course);
  return data;
};
```

Refactor the Kanbas root component so that it uses the **createCourse** client function in the Course client file as shown below. Confirm that new courses are inserted in the courses collection in the database.

*Kanbas/index.js*

```
import { useEffect, useState } from "react";
import * as courseClient from "../Courses/client";
export default function Kanbas() {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  const [course, setCourse] = useState<any>({ ... });

  const addNewCourse = async () => {
    // const newCourse = await userClient.createCourse(course);
    const newCourse = await courseClient.createCourse(course);
    setCourses([...courses, newCourse]);
  };
  ...
  return ( ... );
}
```

## 5.1.3 Deleting Courses from the Database

Refactor the **deleteCourse()** DAO function to delete courses from the database by using the courses model as shown below.

*Kanbas/Courses/dao.js*

```
export function deleteCourse(courseId) {
  return model.deleteOne({ _id: courseId });
}
...
```

In the Courses routes file, refactor the route that deletes courses by adding keywords **async** and **await** in front of the route and **DAO** functions as shown below.

*Kanbas/Courses/routes.js*

```
import * as dao from "../dao.js";
export default function CourseRoutes(app) {
  app.delete("/api/courses/:courseId", async (req, res) => {
    const { courseId } = req.params;
    const status = await dao.deleteCourse(courseId);
    res.send(status);
  });
  ...
}
```

In the React.js application, refactor the **deleteCourse()** client function so that it uses the axios instance with credentials.

*src/Kanbas/Courses/client.ts*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;
export const deleteCourse = async (id: string) => {
  const { data } = await axiosWithCredentials.delete(`${COURSES_API}/${id}`);
  return data;
};
...
```

Refactor the **Kanbas** root component by adding keywords **async** and **await** in front of the **deleteCourse** event handler function and Course client function as shown below. Confirm that courses are removed from the **courses** collection.

*src/Kanbas/index.ts*

```
import { useSelector } from "react-redux";
import { useEffect, useState } from "react";
import * as courseClient from "../Courses/client";
export default function Kanbas() {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  const [courses, setCourses] = useState<any[]>([]);
  const deleteCourse = async (courseId: string) => {
    const status = await courseClient.deleteCourse(courseId);
    setCourses(courses.filter((course) => course._id !== courseId));
  };
  ...
}
```

## 5.1.4 Updating Courses in the Database

In the **Courses DAO**, refactor the **updateCourses** function to update courses in the database with the **model.updateOne()** function as shown below.

*Kanbas/Courses/dao.js*

```
import model from "../model.js";
export function updateCourse(courseId, courseUpdates) {
  return model.updateOne({ _id: courseId }, { $set: courseUpdates });
  // const { courses } = Database;
  // const course = courses.find((course) => course._id === courseId);
  // Object.assign(course, courseUpdates);
  // return course;
}
```

In the **Courses routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

*Kanbas/Courses/routes.js*

```
import * as dao from "../dao.js";
export default function CourseRoutes(app) {
  app.put("/api/courses/:courseId", async (req, res) => {
    const { courseId } = req.params;
    const courseUpdates = req.body;
    const status = await dao.updateCourse(courseId, courseUpdates);
    res.send(status);
  });
  ...
}
```

In the **Courses client** file, refactor the client function to use **axios** with credentials as shown below. Confirm that updating the title of a course is persisted in the database.

*src/Kanbas/Courses/client.ts*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;
export const updateCourse = async (course: any) => {
  const { data } = await axiosWithCredentials.put(`${COURSES_API}/${course._id}`, course);
  return data;
};
```

## 5.2 Persisting Modules in a Database as One to Many Relations with Courses

In **Kanbas**, each course contains several modules, establishing a **one to many relationship**. The relationship is implemented by each module having a field that refers to the course they belong to. This section demonstrates how to use Mongoose to implement **one to many relationships**.

### 5.2.1 Declaring One to Many Relationships

In Mongoose, relationships are implemented by declaring fields of type **ObjectId** and providing the name of the model that is related to. To demonstrate, create the **schema** file below that describes the data structure of **module** documents stored in a collection called **modules**. The **course** field is declared as type **mongoose.Schema.Types.ObjectId** to store a **primary key** value. The **ref** property set to the value **CourseModel** is the same value as the name of the **courses model** stored in the **courses** collection, declared in an earlier section. The **ref** property establishes that the **primary key** stored in **course** refers to a document stored in the **courses** collection, effectively implementing a **one to many** relation.

*Kanbas/Modules/schema.js*

```
import mongoose from "mongoose";
const schema = new mongoose.Schema(
  {
    name: String,
    description: String,
    course: { type: mongoose.Schema.Types.ObjectId, ref: "CourseModel" },
  },
  { collection: "modules" }
);
export default schema;
```

In a **Modules model** file implement a **mongoose model** to interact with the **module** collection in the database. The **model** implements various functions to implement **CRUD** operations such as **find()**, **create()**, **deleteOne()**, **updateOne()**.

*Kanbas/Modules/model.js*

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("ModuleModel", schema);
export default model;
```

### 5.2.2 Creating Modules for a Course

Refactor the **Modules DAO** so that it uses the **mongoose model** to insert a new module into the database as shown below.

*Kanbas/Modules/dao.js*

```
import Database from "../../Database/index.js";
```

```
import model from "../model.js";
export function createModule(module) {
  delete module._id
  return model.create(module);
  // const newModule = { ...module, _id: Date.now().toString() };
  // Database.modules = [...Database.modules, newModule];
  // return newModule;
}
```

In the **Courses routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

*Kanbas/Courses/routes.js*

```
import * as modulesDao from "../Modules/dao.js";
import * as dao from "../dao.js";
export default function CourseRoutes(app) {
  app.post("/api/courses/:courseId/modules", async (req, res) => {
    const { courseId } = req.params;
    const module = {
      ...req.body,
      course: courseId,
    };
    const newModule = await modulesDao.createModule(module);
    res.send(newModule);
  });
  ...
}
```

In the **Courses client** file, refactor the client function to use **axios** with credentials as shown below. Confirm that new modules are created in the database and that the **course** field is set to the **primary key** of the parent **course**.

*src/Kanbas/Courses/client.js*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;
export const createModuleForCourse = async (courseId: string, module: any) => {
  const response = await axiosWithCredentials.post(
    `${COURSES_API}/${courseId}/modules`,
    module
  );
  return response.data;
};
```

## 5.2.3 Retrieving Modules for a Course

Refactor the **Modules DAO** so that it uses the **mongoose model** to retrieve modules for a course from the database as shown below.

*Kanbas/Modules/dao.js*

```
import model from "../model.js";
export function findModulesForCourse(courseId) {
  return model.find({ course: courseId });
  // const { modules } = Database;
  // return modules.filter((module) => module.course === courseId);
}
```

In the **Courses routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

*Kanbas/Courses/routes.js*

```
import * as modulesDao from "../Modules/dao.js";
import * as dao from "../dao.js";
export default function CourseRoutes(app) {
  app.get("/api/courses/:courseId/modules", async (req, res) => {
    const { courseId } = req.params;
    const modules = await modulesDao.findModulesForCourse(courseId);
    res.json(modules);
  });
  ...
}
```

In the **Courses client** file, refactor the client function to use **axios** with credentials as shown below. Confirm that modules are retrieved and displayed from the database for the correct course.

*src/Kanbas/Courses/client.js*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;
export const findModulesForCourse = async (courseId: string) => {
  const response = await axiosWithCredentials.get(`${COURSES_API}/${courseId}/modules`);
  return response.data;
};
```

## 5.2.4 Deleting Modules

Refactor the **Modules DAO** so that it uses the **mongoose model** to delete modules from the database as shown below.

*Kanbas/Modules/dao.js*

```
import model from "../model.js";
export function deleteModule(moduleId) {
  return model.deleteOne({ _id: moduleId });
  // const { modules } = Database;
  // Database.modules = modules.filter((module) => module._id !== moduleId);
}
```

In the **Modules routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

*Kanbas/Modules/routes.js*

```
import * as modulesDao from "../dao.js";
export default function ModuleRoutes(app) {
  app.delete("/api/modules/:moduleId", async (req, res) => {
    const { moduleId } = req.params;
    const status = await modulesDao.deleteModule(moduleId);
    res.send(status);
  });
}
```

In the **Modules client** file, refactor the client function to use **axios** with credentials as shown below. Confirm that modules are removed from the database.

*src/Kanbas/Courses/Modules/client.js*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const MODULES_API = `${REMOTE_SERVER}/api/modules`;
export const deleteModule = async (moduleId) => {
  const response = await axiosWithCredentials.delete(
    `${MODULES_API}/${moduleId}`
  );
}
```

```

);
return response.data;
};

```

## 5.2.5 Updating Modules

Refactor the **Modules DAO** so that it uses the **mongoose model** to update modules in the database as shown below.

*Kanbas/Modules/dao.js*

```

import model from "../model.js";
export function updateModule(moduleId, moduleUpdates) {
  return model.updateOne({ _id: moduleId }, moduleUpdates);
  // const { modules } = Database;
  // const module = modules.find((module) => module._id === moduleId);
  // Object.assign(module, moduleUpdates);
  // return module;
}

```

In the **Modules routes**, refactor the routing function by adding **async** and **await** keywords before the routing and DAO function calls as shown below.

*Kanbas/Modules/routes.js*

```

import * as modulesDao from "../dao.js";
export default function ModuleRoutes(app) {
  ...
  app.put("/api/modules/:moduleId", async (req, res) => {
    const { moduleId } = req.params;
    const moduleUpdates = req.body;
    const status = await modulesDao.updateModule(moduleId, moduleUpdates);
    res.send(status);
  });
  ...
}

```

In the **Modules client** file, refactor the client function to use **axios** with credentials as shown below. Confirm that modules are updated in the database.

*src/Kanbas/Courses/Modules/client.js*

```

import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const MODULES_API = `${REMOTE_SERVER}/api/modules`;
...
export const updateModule = async (module: any) => {
  const { data } = await axiosWithCredentials.put(
    `${MODULES_API}/${module._id}`,
    module
  );
  return data;
};

```

## 5.3 Persisting Enrollments in a Database as Many to Many Relations

In Kanbas, a user can be enrolled in several courses, and a course can have many enrollments. An enrollment establishes a relationship between a user and a course. Since there can be many enrollments where many users can be enrolled (or associated) in many courses, the enrollments relation is referred to as a **many to many relation**.

## 5.3.1 Declaring Enrollments as a Many to Many Relationship

The **Courses model** below, implemented earlier, declares **CourseModel** as the name of the model for **course** documents stored in the **courses** collection. This name can be used to establish relationships between **models** and **collections**.

*Kanbas/Courses/model.js*

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("CourseModel", schema);
export default model;
```

Similarly the **Users model** below declares the name of the model as **UserModel** for **user** documents stored in the **users** collection.

*Kanbas/Users/model.js*

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("UserModel", schema);
export default model;
```

*// Load mongoose library*  
*// Load users schema*  
*// create mongoose model from the schema*  
*// export so it can be used elsewhere*

In a new **schema** file shown below, implement a **many to many Enrollments** relationship that relates **user** and **course** documents stored in the **users** and **courses** collections, referred to by their model names **CourseModel** and **UserModel** respectively.

*Kanbas/Enrollments/schema.js*

```
import mongoose from "mongoose";
const enrollmentSchema = new mongoose.Schema(
{
  course: { type: mongoose.Schema.Types.ObjectId, ref: "CourseModel" },
  user: { type: mongoose.Schema.Types.ObjectId, ref: "UserModel" },
  grade: Number,
  letterGrade: String,
  enrollmentDate: Date,
  status: {
    type: String,
    enum: ["ENROLLED", "DROPPED", "COMPLETED"],
    default: "ENROLLED",
  },
},
{ collection: "enrollments" }
);
export default enrollmentSchema;
```

Create an **Enrollments model** file as shown below, to **CRUD** enrollment documents in an **enrollments** collection.

*Kanbas/Enrollments/model.js*

```
import mongoose from "mongoose";
import schema from "../schema.js";
const model = mongoose.model("EnrollmentModel", schema);
export default model;
```

Create an **Enrollments DAO** file that implements operations that create enrollments, deletes enrollments, and filters enrollments by either a course or user. The following sections describe each of the operations in detail.

*Kanbas/Enrollments/dao.js*

```
import model from "../model.js";
export async function findCoursesForUser(userId) {
```



```

const enrollments = await model.find({ user: userId }).populate("course");
return enrollments.map((enrollment) => enrollment.course);
}
export async function findUsersForCourse(courseId) {
  const enrollments = await model.find({ course: courseId }).populate("user");
  return enrollments.map((enrollment) => enrollment.user);
}
export function enrollUserInCourse(user, course) {
  return model.create({ user, course });
}
export function unenrollUserFromCourse(user, course) {
  return model.deleteOne({ user, course });
}

```

## 5.3.2 Retrieving Courses for Enrolled Users

**Enrollments** establish a *many to many relationship* between **users** and **courses**. A common operation consists of finding which documents in one collection are related to documents in the other collection. For instance, given a particular user we would like to determine which courses are related to that user, e.g., which courses is a user enrolled in. The **findCoursesForUser()** function below retrieves the **enrollment** documents for a given **user**. The **enrollments** documents would contain the **primary keys** for the **user** and **course** documents being referenced. The **populate()** function tells **mongoose** to use the value of the **primary keys** to fetch the actual document referenced by the **key**. The **populate("course")** function replaces the **course primary key** value in the **enrollments** document with the actual **course** document from the **courses** collection corresponding to the **key's** value. The **enrollments.map()** operation unwraps the **enrollments** array and returns a new array with just the **course** objects.

*Kanbas/Enrollments/dao.js*

```

import model from "../model.js";
export async function findCoursesForUser(userId) {
  const enrollments = await model.find({ user: userId }).populate("course");
  return enrollments.map((enrollment) => enrollment.course);
}
...

```

In the **Users routes**, implement a route function **findCoursesForUser** to retrieve courses for a given user. First make sure there's a logged in user. If the logged in user's role is **ADMIN**, then respond with all the courses. Use the user's ID to retrieve the user's courses using the **findCoursesForUser** in the **DAO** implemented above.

*Kanbas/Users/routes.js*

```

import * as dao from "../dao.js";
import * as courseDao from "../Courses/dao.js";
import * as enrollmentsDao from "../Enrollments/dao.js";
export default function UserRoutes(app) {
  const findCoursesForUser = async (req, res) => {
    const currentUser = req.session["currentUser"];
    if (!currentUser) {
      res.sendStatus(401);
      return;
    }
    if (currentUser.role === "ADMIN") {
      const courses = await courseDao.findAllCourses();
      res.json(courses);
      return;
    }
    let { uid } = req.params;
    if (uid === "current") {
      uid = currentUser._id;
    }
    const courses = await enrollmentsDao.findCoursesForUser(uid);
    res.json(courses);
  };
  app.get("/api/users/:uid/courses", findCoursesForUser);
  ...
}

```

```
}
```

In the React.js project, create **findCoursesForUser()** client function to request the **courses** for a given **user ID**.

*src/Kanbas/Account/client.ts*

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
export const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
export const USERS_API = `${REMOTE_SERVER}/api/users`;
export const findCoursesForUser = async (userId: string) => {
  const response = await axiosWithCredentials.get(`${USERS_API}/${userId}/courses`);
  return response.data;
};
```

In the **Kanbas** root component, implement an **enrolling** boolean state variable that toggles true/false to either display all courses or only the courses the current user is enrolled in. Implement event handler functions **findCoursesForUser** and **fetchCourses** as shown below to only fetch the courses a user is enrolled in or, also fetch all the courses. If enrolling, use both sets of courses to toggle an **enrolled** flag in the courses that can be used to display an **Enroll/Unenroll** button for each course. Pass **enrolling** and **setEnrolling** to the **Dashboard** so that it knows how to render the courses.

*src/Kanbas/index.tsx*

```
import { useSelector } from "react-redux";
import { useEffect, useState } from "react";
import * as courseClient from "../Courses/client";
import * as userClient from "../Account/client";
export default function Kanbas() {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  const [courses, setCourses] = useState<any[]>([]);
  const [enrolling, setEnrolling] = useState<boolean>(false);
  const findCoursesForUser = async () => {
    try {
      const courses = await userClient.findCoursesForUser(currentUser._id);
      setCourses(courses);
    } catch (error) {
      console.error(error);
    }
  };
  const fetchCourses = async () => {
    try {
      const allCourses = await courseClient.fetchAllCourses();
      const enrolledCourses = await userClient.findCoursesForUser(
        currentUser._id
      );
      const courses = allCourses.map((course: any) => {
        if (enrolledCourses.find((c: any) => c._id === course._id)) {
          return { ...course, enrolled: true };
        } else {
          return course;
        }
      });
      setCourses(courses);
    } catch (error) {
      console.error(error);
    }
  };
  ...
  useEffect(() => {
    if (enrolling) {
      fetchCourses();
    } else {
      findCoursesForUser();
    }
  }, [currentUser, enrolling]);
  ...
  return (
    ...
    <Dashboard courses={courses} course={course} setCourse={setCourse}
      addNewCourse={addNewCourse} deleteCourse={deleteCourse} updateCourse={updateCourse}
    >
```

```

    enrolling={enrolling} setEnrolling={setEnrolling}/>
  ...
};}

```

In the **Dashboard** screen, use the **enrolling** and **setEnrolling** to implement a new button that toggles the **enrolling** boolean state variable. If **enrolling** is true, show a **Enroll/Unenroll** button for each course. If the **enrolled** flag is true, show **Unenrolled** and **Enroll** otherwise.

*src/Kanbas/Dashboard.ts*

```

...
export default function Dashboard({ ..., enrolling, setEnrolling }
: { ...; enrolling: boolean; setEnrolling: (enrolling: boolean) => void; }) {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  ...
  return (
    <div id="wd-dashboard">
      <h1 id="wd-dashboard-title">
        Dashboard
        <button onClick={() => setEnrolling(!enrolling)} className="float-end btn btn-primary" >
          {enrolling ? "My Courses" : "All Courses"}
        </button>
      </h1>
      ...
      {courses.map((course) => (
        ...
        <div className="card-body">
          <h5 className="wd-dashboard-course-title card-title">
            {enrolling && (
              <button className={`btn ${ course.enrolled ? "btn-danger" : "btn-success" } float-end`} >
                {course.enrolled ? "Unenroll" : "Enroll"}
              </button>
            )}
            {course.name}
          </h5>
          ...
        </div>
      ))}
      ...
    </div>
  );}
};}

```

### 5.3.3 Enrolling / Unenrolling

In the **Enrollments DAO**, implement **enrollUserInCourse** and **unenrollUserFromCourse** functions as shown below. The **enrollUserInCourse** function inserts a new **enrollment** document in the **enrollments** collection creating a relation between a user and the course they are enrolled in. The **unenrollUserFromCourse** function deletes an existing **enrollment** document from the **enrollments** collection, removing the relation between a user and the course they were enrolled in.

*Kanbas/Enrollments/dao.js*

```

import model from "../model.js";
export async function findCoursesForUser(userId) {
  const enrollments = await model.find({ user: userId }).populate("course");
  return enrollments.map((enrollment) => enrollment.course);
}
export function enrollUserInCourse(user, course) {
  return model.create({ user, course });
}
export function unenrollUserFromCourse(user, course) {
  return model.deleteOne({ user, course });
}

```

In the **Users routes** implement **post** and **delete** routes that create or removes an enrollment using the corresponding **DAO** functions.

#### Kanbas/Users/routes.js

```
import * as enrollmentsDao from "../Enrollments/dao.js";
export default function UserRoutes(app) {
  const findCoursesForUser = async (req, res) => { ... };
  const enrollUserInCourse = async (req, res) => {
    let { uid, cid } = req.params;
    if (uid === "current") {
      const currentUser = req.session["currentUser"];
      uid = currentUser._id;
    }
    const status = await enrollmentsDao.enrollUserInCourse(uid, cid);
    res.send(status);
  };
  const unenrollUserFromCourse = async (req, res) => {
    let { uid, cid } = req.params;
    if (uid === "current") {
      const currentUser = req.session["currentUser"];
      uid = currentUser._id;
    }
    const status = await enrollmentsDao.unenrollUserFromCourse(uid, cid);
    res.send(status);
  };
  app.post("/api/users/:uid/courses/:cid", enrollUserInCourse);
  app.delete("/api/users/:uid/courses/:cid", unenrollUserFromCourse);
  app.get("/api/users/:uid/courses", findCoursesForUser);
  ...
}
```

In the React.js application, create client functions **enrollIntoCourse** and **unenrollFromCourse** that request the server to enroll or unenroll a user from a course. Encode the primary keys of the **user** and **course** as part of the path.

#### src/Kanbas/Account/client.ts

```
import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
export const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
export const USERS_API = `${REMOTE_SERVER}/api/users`;
export const findCoursesForUser = async (userId: string) => { ... }
export const enrollIntoCourse = async (userId: string, courseId: string) => {
  const response = await axiosWithCredentials.post(`${USERS_API}/${userId}/courses/${courseId}`);
  return response.data;
};
export const unenrollFromCourse = async (userId: string, courseId: string) => {
  const response = await axiosWithCredentials.delete(`${USERS_API}/${userId}/courses/${courseId}`);
  return response.data;
};
```

In the **Kanbas** root component, implement **updateEnrollment()** event handler which calls the client functions **enrollIntoCourse** or **unenrollFromCourse** based on whether enrolled is true or false, as shown below. Pass the **updateEnrollment()** function to the **Dashboard** screen.

#### src/Kanbas/index.ts

```
import * as userClient from "../Account/client";
export default function Kanbas() {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  const [enrolling, setEnrolling] = useState<boolean>(false);
  const findCoursesForUser = async () => { ... };
  const updateEnrollment = async (courseId: string, enrolled: boolean) => {
    if (enrolled) {
      await userClient.enrollIntoCourse(currentUser._id, courseId);
    } else {
      await userClient.unenrollFromCourse(currentUser._id, courseId);
    }
  }
  setCourses(
    courses.map((course) => {
      if (course._id === courseId) {
        return { ...course, enrolled: enrolled };
      } else {

```

```

        return course;
    }
  })
  );
};

const fetchCourses = async () => { ... }
...
useEffect(() => { ... }
...
return (
  ...
  <Dashboard courses={courses} course={course} setCourse={setCourse}
    addNewCourse={addNewCourse} deleteCourse={deleteCourse} updateCourse={updateCourse}
    enrolling={enrolling} setEnrolling={setEnrolling} updateEnrollment={updateEnrollment}/>
  ...
);

```

In the **Dashboard** screen implement an **onClick** event handler that calls **updateEnrollment()** with the **course's ID** and toggles the **course's enrolled** flag, causing the user to enroll or unenroll accordingly. Confirm that users can enroll and unenroll from any course.

*src/Kanbas/Dashboard.ts*

```

...
export default function Dashboard({ ..., enrolling, setEnrolling, updateEnrollment }
: { ...; enrolling: boolean; setEnrolling: (enrolling: boolean) => void;
  updateEnrollment: (courseId: string, enrolled: boolean) => void }) {
  const { currentUser } = useSelector((state: any) => state.accountReducer);
  ...
  return (
    <div id="wd-dashboard">
      ...
      {courses.map((course) => (
        ...
        <div className="card-body">
          <h5 className="wd-dashboard-course-title card-title">
            {enrolling && (
              <button onClick={(event) => {
                event.preventDefault();
                updateEnrollment(course._id, !course.enrolled);
              }}
              className={`btn ${course.enrolled ? "btn-danger" : "btn-success"} float-end`} >
                {course.enrolled ? "Unenroll" : "Enroll"}
            </button>
          )}
          {course.name}
        </h5>
        ...
      </div>
    )]}
    ...
  </div>
  ...
);

```

## 5.3.4 Enrolling the Author

When a user creates a course, they should be automatically be enrolled in the course, otherwise they would not see the course in their **Dashboard** when they signin. In the **Courses routes**, in the route that creates new courses, use the **IDs** of the signed in user and the new course to enroll the user in the new course. Use the code below as a guide. Confirm that users that create courses are enrolled in the new course.

*Kanbas/Courses/routes.js*

```

import * as enrollmentsDao from "../Enrollments/dao.js";

export default function CourseRoutes(app) {
  app.post("/api/courses", async (req, res) => {

```

```

const course = await dao.createCourse(req.body);
const currentUser = req.session["currentUser"];
if (currentUser) {
  await enrollmentsDao.enrollUserInCourse(currentUser._id, course._id);
}
res.json(course);
});

```

## 5.3.5 Retrieving a Course's Students (On Your Own)

The **Users** screen implement earlier uses the **PeopleTable** to display all the users in the database. The **People** route implemented earlier under the **Courses** screen, should display the users in the current course. Reimplement the **People** route so that the **PeopleTable** only displays the users that are enrolled in a particular course when navigating to the **People** link. In the **Courses routes** use the **enrollments DAO findUsersForCourse** function to retrieve the users enrolled in a course.

*Kanbas/Courses/routes.ts*

```

import * as enrollmentsDao from "../Enrollments/dao.js";
export default function CourseRoutes(app) {
  const findUsersForCourse = async (req, res) => {
    const { cid } = req.params;
    const users = await enrollmentsDao.findUsersForCourse(cid);
    res.json(users);
  };
  app.get("/api/courses/:cid/users", findUsersForCourse);
  ...
}

```

In **Courses client** implement **findUsersForCourse()** client function to retrieve the users for a given course. Use the **Courses routes** and **client** to display the users enrolled in a course when navigating to a course's **People** route.

*src/Kanbas/Courses/client.ts*

```

import axios from "axios";
const axiosWithCredentials = axios.create({ withCredentials: true });
const REMOTE_SERVER = process.env.REACT_APP_REMOTE_SERVER;
const COURSES_API = `${REMOTE_SERVER}/api/courses`;

export const findUsersForCourse = async (courseId: string) => {
  const response = await axios.get(`${COURSES_API}/${courseId}/users`);
  return response.data;
};

```

## 5.4 Assignments (On Your Own)

Implement **schema**, **model**, **DAO**, **routes**, and **client** files so that the **Assignments** and **AssignmentsEditor** screens display assignments stored in a database. Users should be able to display assignments in a course, create new assignments, update assignments, and delete existing assignments.

# 6 Deliverables

As a deliverable, make sure you complete all the lab exercises, mongoose schemas, models, DAOs, React components, and they behave as described. For both the React and Node repositories, all your work should be done in a branch called **a6**. When done, add, commit and push both branches to their respective GitHub repositories. Deploy the new branches to **Netlify** and **Render** (or **Heroku**) and confirm they integrate. If you are using **Render**, it does not create a separate branch deployment like **Netlify** so you'll have to deploy to an entirely different Web service so that you don't trample the previous

assignment while the TAs are still grading. The Node server application running on Render will need to be configured to interact with the **a6 Netlify** branch deployment. All the exercises should work remotely just as well as locally. The Kanbas Dashboard should display the courses and modules from the database. As a deliverable in **Canvas**, submit the URL to the **a6** branch deployment of your React.js application running on Netlify.